Docker Commands Cheatsheet

Images, containers, execution, cleanup, troubleshooting, and hands-on practice

Notation: `IMAGE` = Docker image name; `CONTAINER` = container name or ID.

1. Images vs. Containers

Image: A read-only template used to create containers.
Container: A running or stopped instance created from an image.

```
docker pull → IMAGE → docker create → CREATED → docker start → RUNNING → docker stop →
STOPPED → docker rm
```

Shortcut: `docker run` normally combines image retrieval (if needed), container creation, and starting the container.

2. Download and Manage Images

docker pull

```
docker pull IMAGE[:TAG]
```

Option	Purpose
`-q, --quiet`	Suppress verbose output.
`--platform PLATFORM`	Pull an image for a specific platform, such as linux/amd64 or linux/arm64.
`--all-tags`	Download all tags of the repository.

```
docker pull ubuntu
docker pull ubuntu:24.04
docker pull python:3.12-slim
docker pull --platform linux/amd64 ubuntu
```

Note: If no tag is specified, Docker generally uses the latest tag.

docker images

```
docker images [OPTIONS] [REPOSITORY[:TAG]]
```

Option	Purpose
`-a, --all`	Show all images, including intermediate layers.
`-q, --quiet`	Display only image IDs.
`--no-trunc`	Show full image IDs.
`--digests`	Show content digests.
`--filter FILTER`	Filter results, for example dangling=true.
`--format FORMAT`	Customize output using Go templates.

```
docker images
docker images ubuntu
docker images -a
docker images -q
docker images --filter dangling=true
```

docker rmi / docker image rm

```
docker rmi [OPTIONS] IMAGE [IMAGE...]
```

Option	Purpose
`-f, --force`	Force removal where applicable.
`--no-prune`	Do not delete untagged parent images.

```
docker rmi ubuntu:latest
docker rmi IMAGE_ID
docker rmi -f IMAGE_ID
```

3. List Containers

docker ps

```
docker ps [OPTIONS]
```

Option	Purpose
`-a, --all`	Show running and stopped containers.
`-q, --quiet`	Show only container IDs.
`-l, --latest`	Show the latest created container.
`-n N, --last N`	Show the last N created containers.
`-s, --size`	Display container size.
`-f, --filter FILTER`	Filter by status, name, ID, and more.
`--no-trunc`	Show full IDs and untruncated output.
`--format FORMAT`	Customize output columns.

```
docker ps
docker ps -a
docker ps -q
docker ps -f "status=exited"
docker ps -f "name=cont1"
```

Note: docker ps = running; docker ps -a = all; docker ps -q = IDs only; docker ps -aq = IDs of all containers.

Linux ps commands

Option	Purpose
`ps`	Processes associated with the current terminal/session.
`ps -a`	Processes with terminals, generally excluding session leaders without a terminal.
`ps -A`	All processes.
`ps x`	Include processes without a controlling terminal.
`ps u`	User-oriented format for selected processes.
`ps aux`	All processes in user-oriented format, including those without a terminal.
`ps aux | grep cont1`	Search the process listing for text matching cont1.

```
ps aux
ps aux | grep cont1
```

4. Create, Start, and Stop Containers

docker create

```
docker create [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Option	Purpose
`--name NAME`	Assign a custom container name.
`-p HOST:CONTAINER`	Publish a container port on a host port.
`-P, --publish-all`	Publish all exposed ports to random host ports.
`-e KEY=VALUE, --env`	Set environment variables.
`-v HOST:CONTAINER`	Mount a volume or bind mount.
`--network NETWORK`	Connect to a specified network.
`-w, --workdir`	Set the working directory.
`--restart POLICY`	Configure restart behavior.
`--entrypoint COMMAND`	Override the image entrypoint.
`--rm`	Remove the container automatically when it exits.

```
docker create --name cont1 ubuntu
docker create --name web -p 8080:80 nginx
docker create -it --name dev ubuntu bash
```

Note: A created container is not running yet.

docker start

```
docker start [OPTIONS] CONTAINER [CONTAINER...]
```

Option	Purpose
`-a, --attach`	Attach the container's standard streams.
`-i, --interactive`	Attach stdin.
`-q, --quiet`	Suppress output.

```
docker start cont1
docker start -ai cont1
```

docker stop

```
docker stop [OPTIONS] CONTAINER [CONTAINER...]
```

Option	Purpose
`-t SECONDS, --timeout`	Seconds to wait before forceful termination.
`-s SIGNAL, --signal`	Signal to send instead of the default stop signal.

```
docker stop cont1
docker stop -t 30 cont1
```

Note: docker kill stops a container immediately using a kill signal by default.

5. Execute Commands Inside Containers

docker exec

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

Option	Purpose
`-i, --interactive`	Keep stdin open.
`-t, --tty`	Allocate a pseudo-terminal (TTY).
`-it`	Interactive terminal; commonly used for Bash or another shell.
`-d, --detach`	Run the command in the background.
`-e KEY=VALUE, --env`	Set environment variables for the executed command.
`-w, --workdir`	Set the working directory.
`-u, --user`	Run as a specified user or UID.
`--privileged`	Give the exec process extended privileges.
`--env-file FILE`	Read environment variables from a file.

```
docker exec cont1 ls
docker exec cont1 pwd
docker exec -it cont1 bash
docker exec -it cont1 sh
docker exec -u root -it cont1 bash
```

Note: -t allocates a pseudo-terminal; -i keeps standard input open; -it combines both. docker exec requires a running container.

6. Run Containers

docker run

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Option	Purpose
`--name NAME`	Set the container name.

Option	Purpose
`-d, --detach`	Run in the background.
`-it`	Interactive terminal.
`-i, --interactive`	Keep stdin open.
`-t, --tty`	Allocate a TTY.
`--rm`	Automatically remove the container when it exits.
`-p HOST:CONTAINER`	Publish a container port on a host port.
`-P, --publish-all`	Publish all exposed ports to random host ports.
`-e KEY=VALUE, --env`	Set environment variables.
`--env-file FILE`	Load environment variables from a file.
`-v HOST:CONTAINER`	Mount a volume or bind mount.
`--mount ...`	Explicitly configure a mount.
`--network NETWORK`	Select the container network.
`--restart POLICY`	Configure automatic restart.
`-w, --workdir`	Set the working directory.
`--entrypoint COMMAND`	Override the image entrypoint.
`--platform PLATFORM`	Select the target platform.
`--user USER`	Run as a specified user.
`--hostname NAME`	Set the container hostname.
`--memory LIMIT`	Limit memory usage.
`--cpus NUMBER`	Limit CPU allocation.

```
docker run -it ubuntu bash
docker run -d --name cont1 ubuntu sleep 3600
docker run -d --name web -p 8080:80 nginx
docker run -it python:3.12-slim python
docker run --rm ubuntu cat /etc/os-release
```

Note: --rm removes the container after its main process exits; it does not remove the image.

7. Inspect and Troubleshoot

docker inspect

```
docker inspect [OPTIONS] NAME|ID [NAME|ID...]
```

Option	Purpose
`-f, --format TEMPLATE`	Extract specific fields using a Go template.
`-s, --size`	Include container size information.
`--type TYPE`	Restrict the object type, such as container or image.

```
docker inspect cont1
docker inspect ubuntu
docker inspect -f "{{.State.Status}}" cont1
docker inspect -f "{{.NetworkSettings.IPAddress}}" cont1
```

Note: Useful fields include .Id, .Name, .Config.Image, .State.Status, .State.Pid, .State.ExitCode, .NetworkSettings, .Mounts, and .Config.Env.

docker logs

```
docker logs [OPTIONS] CONTAINER
```

Option	Purpose
`-f, --follow`	Continue streaming new log output.
`--tail N`	Show only the last N lines.

Option	Purpose
`-t, --timestamps`	Include timestamps.
`--since TIME`	Show logs since a timestamp or relative duration.
`--until TIME`	Show logs until a timestamp or relative duration.
`--details`	Show extra log details when supported.

```
docker logs cont1
docker logs -f cont1
docker logs --tail 50 cont1
docker logs -f --tail 20 web
docker logs --since 10m web
```

Note: docker logs primarily reads the container's configured logging output, usually STDOUT and STDERR.

docker help

```
docker help
docker COMMAND --help
```

```
docker run --help
docker exec --help
docker ps --help
docker image --help
```

8. Remove Containers and Clean Up

docker rm

```
docker rm [OPTIONS] CONTAINER [CONTAINER...]
```

Option	Purpose
`-f, --force`	Force removal of running containers.
`-v, --volumes`	Remove anonymous volumes associated with the container.
`-l, --link`	Remove the specified legacy link.

```
docker rm cont1
docker rm cont1 cont2
docker rm -f cont1
docker rm -v cont1
```

docker image prune

```
docker image prune [OPTIONS]
```

Option	Purpose
`-a, --all`	Remove all unused images, not just dangling images.
`-f, --force`	Do not ask for confirmation.
`--filter FILTER`	Apply filters, such as until=24h.

```
docker image prune
docker image prune -a
docker image prune -f
docker image prune --filter 'until=24h'
```

Note: docker image prune -a can remove images not currently referenced by containers.

9. Essential Quick Reference

Command	Purpose
`docker pull`	Download an image.
`docker images`	List local images.
`docker ps`	List running containers.
`docker ps -a`	List all containers.
`docker create`	Create a stopped container.

Command	Purpose
`docker start`	Start a stopped container.
`docker stop`	Gracefully stop a running container.
`docker run`	Create and start a container.
`docker exec`	Execute a command in a running container.
`docker inspect`	View detailed object information.
`docker logs`	View container logs.
`docker rm`	Remove a container.
`docker rmi`	Remove an image.
`docker image prune`	Clean up unused images.

10. Hands-on Practice Workflow

Run the following sequence to practice most of the commands in the course:

1. `docker pull ubuntu:24.04`
Download the Ubuntu image.

2. `docker images`
Verify the image exists locally.

3. `docker create --name cont1 ubuntu:24.04 sleep 3600`
Create a stopped container.

4. `docker ps -a`
Confirm the container is created.

5. `docker start cont1`
Start the container.

6. `docker ps`
Verify that cont1 is running.

7. `docker exec -it cont1 bash`
Open a Bash shell; type exit to leave.

8. `docker logs cont1`
View the container's output.

9. `docker inspect cont1`
Explore its configuration and state.

10. `docker stop cont1`
Stop the container.

11. `docker rm cont1`
Remove the stopped container.

12. `docker rmi ubuntu:24.04`
Remove the local image when no longer needed.

Final Reminders

docker create does not start a container.
docker run creates and starts a container.
docker exec requires a running container.
docker stop does not delete a container.
docker rm removes containers; **docker rmi** removes images.
-it means interactive input plus a pseudo-terminal.
--rm removes the container after it exits.
ps aux inspects Linux processes, not Docker containers directly.

AWS note: These commands work on an EC2 Linux instance once Docker is installed and your user has permission to access the Docker daemon.